# Zip Code Group Project 3.0

TEAM 3

Saint Cloud State University

Spring 2026: CSCI331 Software Systems

Anda Andrew

March 16, 2026

# Project Overview

Zip Code Group Project 3.0 extends Group Project 2.0 by introducing a blocked sequence set file structure, enabling efficient block-based storage, indexed lookup, and dynamic record updates.

This project transitions the system from record-level access to block-level storage. Instead of directly locating individual records by byte offset, the system now locates and processes blocks that contain groups of records.

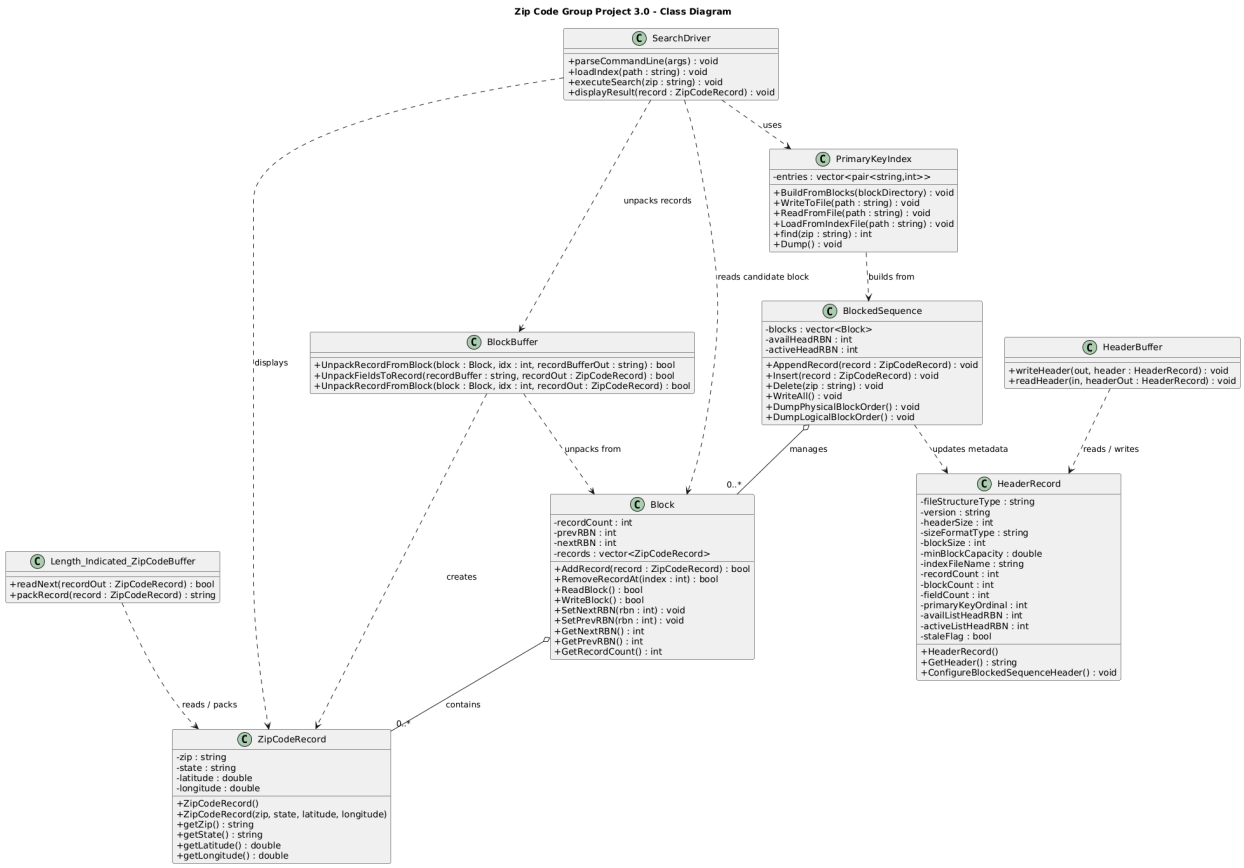
The system is divided into three major components:

- Part I: Generation of a blocked sequence set from the length-indicated file
- Part II: Construction of a block-level primary key index
- Part III: Implementation of insertion and deletion workflows with structural updates

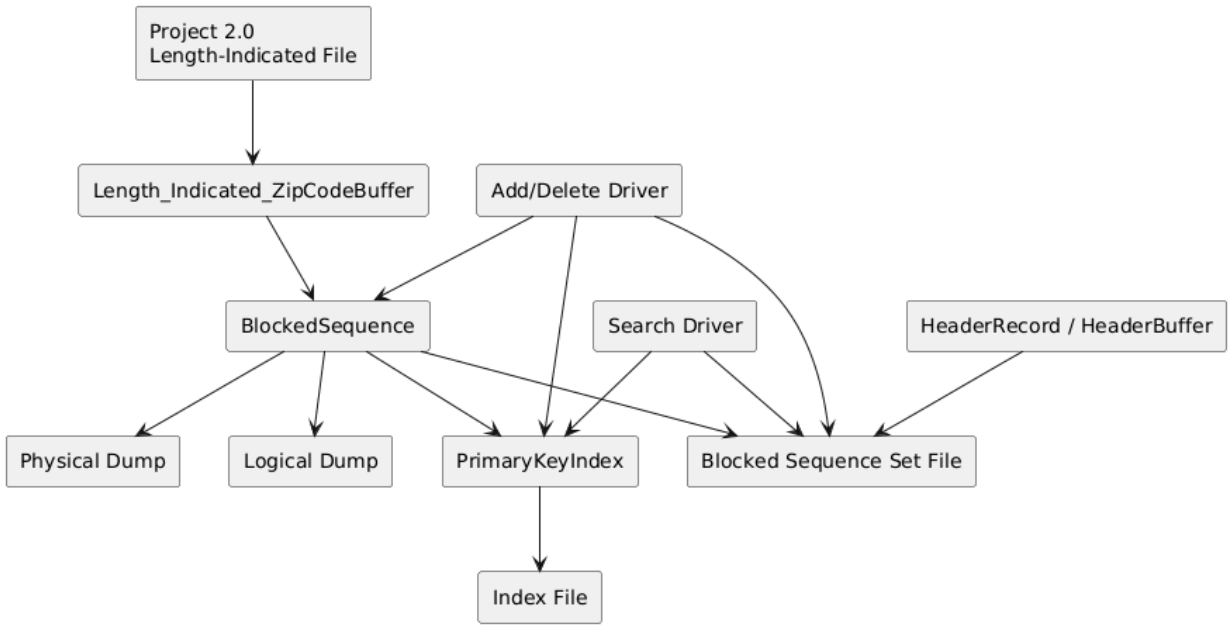
The design maintains sequential processing where possible and ensures that the full dataset is never loaded into memory during search operations.

# Requirements Summary

- Use the Project 2.0 length-indicated file as input
- Generate a blocked sequence set file with a header record
- Use fixed block size (default 512 bytes)
- Store complete records within each block
- Maintain predecessor and successor links between blocks
- Maintain an avail list for reusable blocks
- Provide physical and logical block dumps
- Build a simple index mapping <highest key, RBN>
- Support command-line ZIP lookup flags
- Support record insertion and deletion workflows
- Log structural changes (split, merge, redistribution)
- Document code using Doxygen




## Zip Code Group Project 3.0 - High Level System Flow



# System Architecture

The system is organized into the following logical layers:

- Data Model
- Buffer and Serialization Layer
- Blocked Storage Layer
- Index Layer
- Application Layer

# High-Level Data Flow

## Generation Flow

- Read length-indicated file sequentially
- Group records into blocks
- Write blocks to file
- Generate header metadata
- Build index from blocks

## Search Flow

- Parse command-line input
- Load index into memory
- Locate target block using index
- Read block from disk
- Search records within a block

## Add/Delete Flow

- Locate block using index
- Insert or delete record
- Handle split, redistribution, or merge
- Update block links and avail list
- Update index if necessary

# Blocked Sequence Set Design

## Header Record

The header record defines how the blocked sequence set file is interpreted and contains metadata such as block size, record count, and index information.

## Block Architecture

Active blocks contain:

- Record count
- Previous block link (RBN)
- Next block link (RBN)
- Sorted records

Avail blocks contain:

- Record count = 0
- Pointer to next avail block

- Unused space cleared

# Class Design

## ZipCodeRecord

Represents a single ZIP code record with accessor methods.

## Length_Indicated_ZipCodeBuffer

Handles reading and writing of length-indicated records.

## HeaderRecord and HeaderBuffer

Manage metadata serialization and deserialization.

## Block

Represents a fixed-size block that stores multiple records and maintains links to adjacent blocks.

## BlockBuffer

Handles unpacking of records from a block into usable objects.

## BlockedSequence

Manages the overall block structure, including insertion, deletion, and traversal.

## PrimaryKeyIndex

Stores mappings from the highest key in block to RBN and supports efficient lookup.

# Memory Constraint Compliance

- Index is loaded into memory
- Only one block is read at a time
- Records are processed individually
- Full dataset is never loaded into memory

# Testing Strategy

- Verify header and block structure
- Test valid and invalid searches
- Test insert and delete scenarios
- Verify logs and index updates

# Deliverables

- Design document
- Test document
- User guide
- Data files
- Index file
- Doxygen documentation
- Source code

- Execution script

# Conclusion

Project 3.0 extends the system into a block-based storage architecture, enabling efficient data organization, scalable lookup, and structured update operations